Lab 4 NVIC and EXTI

Objectives

1) Students should be able to understand how interrupt works.
2) Students should be able to use external interrupt.

1. Priority Interrupt

The interrupt is the mechanism that requires microcontroller unit (MCU) temporarily halts the current execution and then handles the incoming interrupt signal such as external interrupt (EXTI) from GPIO. After handling the interrupt, MCU resumes the halting execution.

Nested Vectored Interrupt Controller (NVIC) is the microcontroller module that controls and responses to the interrupt signal. MCU can handle the interrupt from several sources however the interrupt priority for each source must be managed beforehand. After proper interrupt priority is set, NVIC can handle when several interrupts occur simultaneously or when the new interrupt occurs while handling the previous interrupt.

Cortex M7 has been designed to use an 8-bit priority field to configure the priority for each interrupt however each manufacturer can customize this field size for example STM32F767 from STMicroelectronics uses only 4 bits. The 4-bit priority field is separated into 2 parts that are `PreemptionPriority` and `SubPriority`. As a result, interrupt priority is categorized into 5 groups labelled as `NVIC_PriorityGroup_0` to `NVIC_PriorityGroup_4`. Table 1.1 shows the possible value of `PreemptionPriority` and `SubPriority` in each group.

Table 1.1 shows the possible value of `PreemptionPriority` and `SubPriority` for each group.

NVIC_PriorityGroup	**PreemptionPriority**		**SubPriority**	
	Number of bits	Possible values	Number of bits	Possible values
NVIC_PriorityGroup_0	0	0	4	0-15
NVIC_PriorityGroup_1	1	0-1	3	0-7
NVIC_PriorityGroup_2	2	0-3	2	0-3
NVIC_PriorityGroup_3	3	0-7	1	0-1
NVIC_PriorityGroup_4	4	0-15	0	0

The lower value of `PreemptionPriority` indicates the greater priority so if any interrupt signal has `PreemptionPriority` value 0, this means it has the most priority. Furthermore, the interrupt signal

that has lower `PreemptionPriority` value (greater priority) can interrupt the interrupt signal that is currently handling by MCU but has greater `PreemptionPriority` value (lower priority).

If there are 2 or more interrupt signals that have the same `PreemptionPriority` value pending or waiting to be handled, interrupt signal that has lower `SubPriority` value (more important) will be selected to handle first.

2. Interrupt and EXTI Configuration in STM32CubeMX

This section provides the information for setting both NVIC and EXTI. Getting started with tab Pinout in STM32CubeMX and select `GPIO_EXTI13` at PC13 pin. This will configure B1 button on PC13 pin to function as EXTI instead of GPIO input as shown in Fig 2.1.

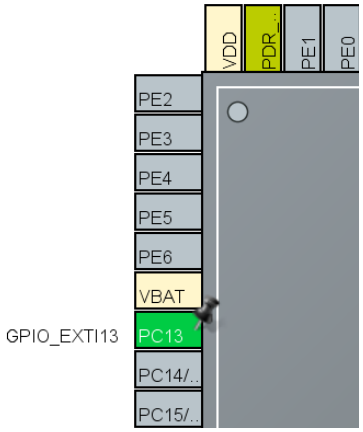


Fig 2.1 shows how to configure PC13 pin to function as EXTI13.

Then choose NVIC group 1 that has 1 bit for `PreemptionPriority` and 3 bits for `SubPriority` and set priority for EXTI13 as shown in Fig 2.2. After that, configure PC13 pin to generate interrupt signal when detects the rising edge and to be floating input pin as shown in Fig 2.3.

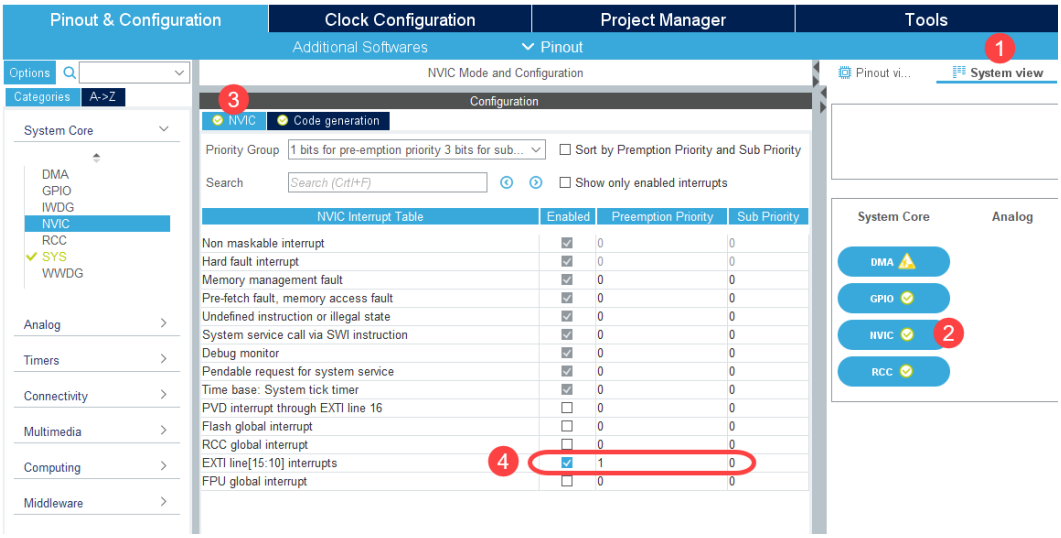


Fig 2.2 shows `NVIC_PriorityGroup_1` configuration.

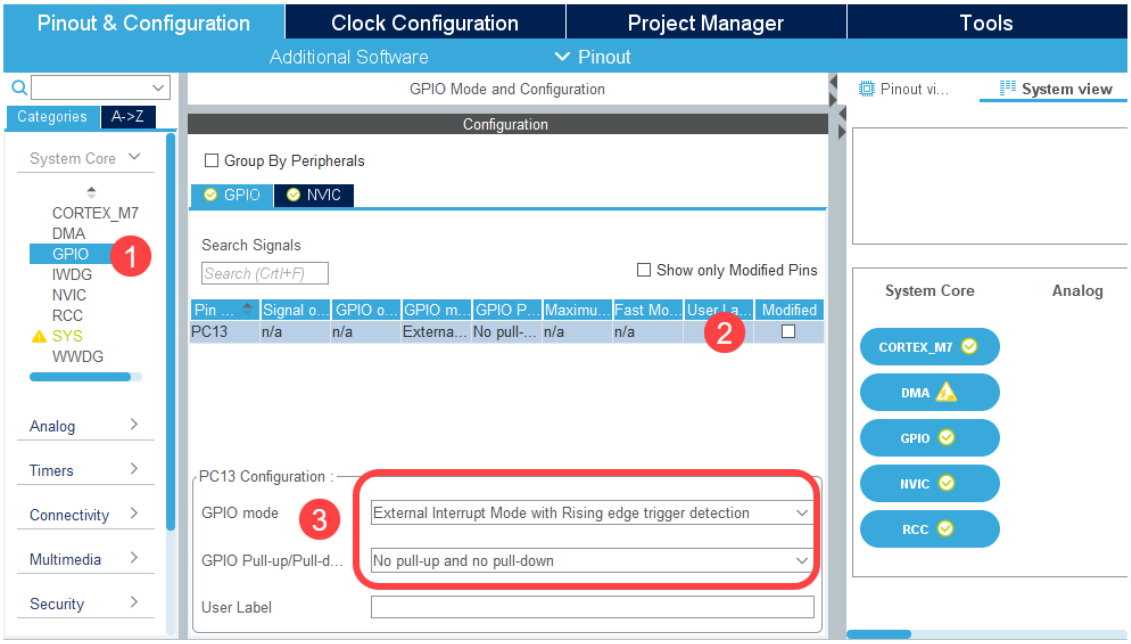


Fig 2.3 shows configuration of PC13 pin to function as EXTI13

3. Explanation

Generated code from STM32CubeMX that is used to configure NVIC is placed in `HAL_MspInit()` function in `stm32f7xx_hal_msp.c` file as shown in Fig 3.1.

`PreemptionPriority` and `SubPriority` configuration for PC13 pin is in `MX_GPIO_Init()` function in `gpio.c` file as shown in Fig 3.2.

```
void HAL_MspInit(void)
{
  /* USER CODE BEGIN MspInit 0 */

  /* USER CODE END MspInit 0 */

  __HAL_RCC_SYSCFG_CLK_ENABLE();
  __HAL_RCC_PWR_CLK_ENABLE();

  HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_1);

  /* System interrupt init*/

  /* USER CODE BEGIN MspInit 1 */

  /* USER CODE END MspInit 1 */
}
```

Fig 3.1 shows the configuration for priority group in `HAL_MspInit()` function.

```
void MX_GPIO_Init(void)
{

  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();

  /*Configure GPIO pin : PC13 */
  GPIO_InitStruct.Pin = GPIO_PIN_13;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

  /* EXTI interrupt init*/
  HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);
  HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);

}
```

Fig 3.2 shows the configuration for PC13 pin to function as EXTI13 in `MX_GPIO_Init()` function.

Function MX_GPIO_init ( )

- is generated from STM32CubeMX and is used to configure GPIO module on MCU.
- First, enable clock signal to GPIOC (B1 button on PC13) using the following statement:

```
__GPIOC_CLK_ENABLE();
```

- Configure PC13 to function as EXTI13. It is set to be floating input pin and generates interrupt signal when detects the rising edge signal on the pin (B1 button is push).

```
GPIO_InitStruct.Pin   = GPIO_PIN_13;
GPIO_InitStruct.Mode  = GPIO_MODE_IT_RISING;
GPIO_InitStruct.Pull  = GPIO_NOPULL;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
```

- Configure EXTI13 to have `PreemptionPriority = 1 and SubPriority = 0.`

```
HAL_NVIC_SetPriority(EXTI15_10_IRQn, 1, 0);
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
```

4. EXTI

External interrupt (EXTI) is microcontroller module that detects digital signal from GPIO pins. EXTI generates interrupt signal to NVIC when encounter predefined condition that is the detection for rising edge, falling edge, or both. Fig 4.1 shows EXTI structure and Fig 4.2 shows the connection between GPIO and EXTI modules. In addition, GPIO pins that have the same pin number regardless of GPIO port share the same EXTI signal for example PA0, PB0 through PG0 pins have the same EXTI number that is EXTI0 as shown in Fig 4.2.

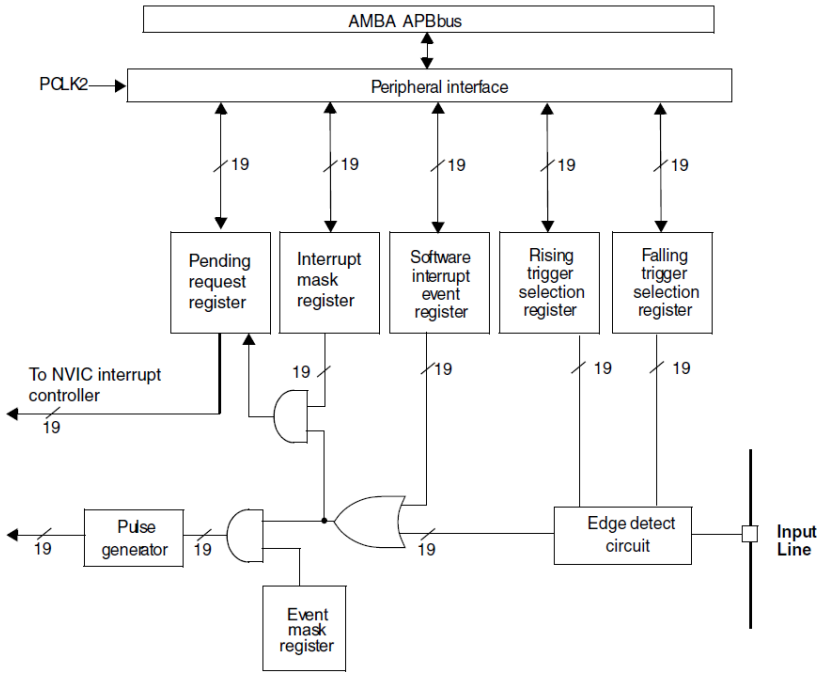


Fig 4.1 shows EXTI structure.

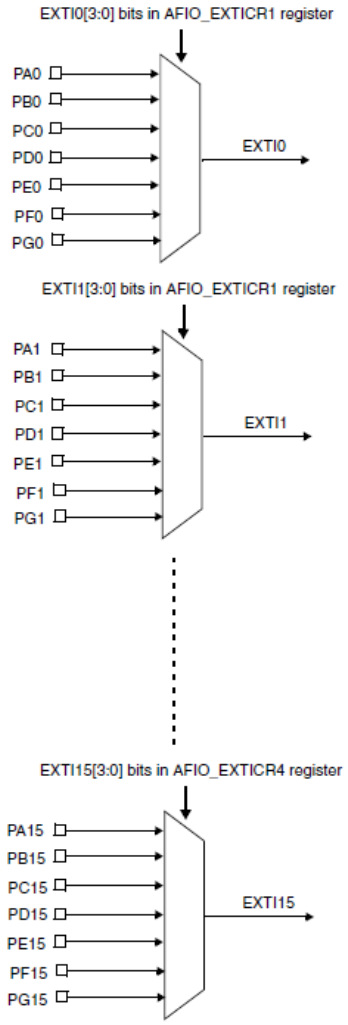


Fig 4.2 shows the connection between GPIO and EXTI modules.

5. Interrupt Service Routine

Interrupt service routine (ISR) is a subroutine that responses to interrupt signal. When processor core inside MCU receives interrupt signal from NVIC, processor core will temporarily halt the current execution and call ISR responded for the interrupt. The location for ISR is stored in the vector table. After ISR is completed, processor core will resume the halting execution.

For example, if configuring NVIC and EXTI according to Fig 2.1, Fig 2.2, and Fig 2.3, B1 button on PC13 pin will generate EXTI13 interrupt signal when it is push. When processor core receives EXTI13 interrupt signal, it will halt the current execution and call `EXTI15_10_IRQHandler()` function which is the ISR for EXTI13.

`EXTI15_10_IRQHandler()` function in `stm32f7xx_it.c` file is ISR that responses to `EXTI15_10_IRQn` interrupt that covers EXTI10 to EXTI15. The location of this function is specified in vector table declared in `startup_stm32f767zitx.s` assembly file as shown in Fig 5.1. Other ISR names for each EXTI number can be shown in Table 5.1.

```
/* External Interrupts */
.word     WWDG_IRQHandler               /* Window WatchDog              */
.word     PVD_IRQHandler                /* PVD through EXTI Line detection */
.word     TAMP_STAMP_IRQHandler         /* Tamper and TimeStamps through the EXTI line */
.word     RTC_WKUP_IRQHandler           /* RTC Wakeup through the EXTI line */
.word     FLASH_IRQHandler              /* FLASH                        */
.word     RCC_IRQHandler                /* RCC                          */
.word     EXTI0_IRQHandler              /* EXTI Line0                   */
.word     EXTI1_IRQHandler              /* EXTI Line1                   */
.word     EXTI2_IRQHandler              /* EXTI Line2                   */
.word     EXTI3_IRQHandler              /* EXTI Line3                   */
.word     EXTI4_IRQHandler              /* EXTI Line4                   */
.word     DMA1_Stream0_IRQHandler       /* DMA1 Stream 0                */
.word     DMA1_Stream1_IRQHandler       /* DMA1 Stream 1                */
```

Fig 5.1 shows vector table definition

Table 5.1 shows ISR names for each EXTI number.

EXTI Number	Interrupt Name	ISR Name	Note
EXTI0	EXTI0_IRQn	EXTI0_IRQHandler	-
EXTI1	EXTI1_IRQn	EXTI1_IRQHandler	-
EXTI2	EXTI2_IRQn	EXTI2_IRQHandler	-
EXTI3	EXTI3_IRQn	EXTI3_IRQHandler	-
EXTI4	EXTI4_IRQn	EXTI4_IRQHandler	-
EXTI5 – EXTI9	EXTI9_5_IRQn	EXTI9_5_IRQHandler	EXTI5 to EXTI9 use the same ISR.
EXTI10 – EXTI15	EXTI15_10_IRQn	EXTI15_10_IRQHandler	EXTI10 to EXTI15 use the same ISR.

Fig 5.2 shows ISR implementation of `EXTI15_10_IRQn` which includes `EXTI13`. This function is called when B1 button is push. This function toggles LD1 LED on PB0 pin. `HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13)` function is used to inspect the interrupt, clear interrupt pending bit, and invoke callback function as shown in Fig 5.3.

```
void EXTI15_10_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI15_10_IRQn 0 */

  /* USER CODE END EXTI15_10_IRQn 0 */
  HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13);
  /* USER CODE BEGIN EXTI15_10_IRQn 1 */
  HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
  /* USER CODE END EXTI15_10_IRQn 1 */
}
```

Fig 5.2 shows interrupt service routine of `EXTI15_10_IRQn`.

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
  /* EXTI line interrupt detected */
  if(__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != RESET)
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Callback(GPIO_Pin);
  }
}
```

Fig 5.3 shows the implementation of `HAL_GPIO_EXTI_IRQHandler()` function.

6. Callback Function

Callback function is the function invoked inside ISR to handle the interrupt more sophisticatedly such as separation handling between success and error interrupt. In Fig 6.1, there is only 1 callback function for EXTI that is `HAL_GPIO_EXTI_Callback()`.

Furthermore, there are 2 callback functions for UART receive interrupt that are `HAL_UART_RxCpltCallback()` and `HAL_UART_ErrorCallback()` functions as shown in Fig 6.2. The first callback function is invoked when the UART receive is success otherwise the second one is invoked instead.

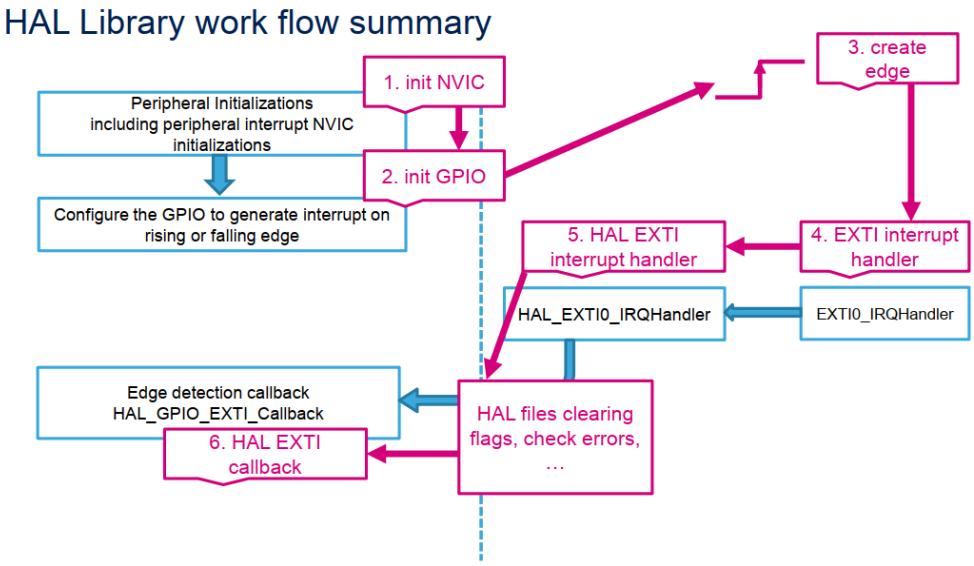


Fig 6.1 shows interrupt handling process of `EXTI0` module.

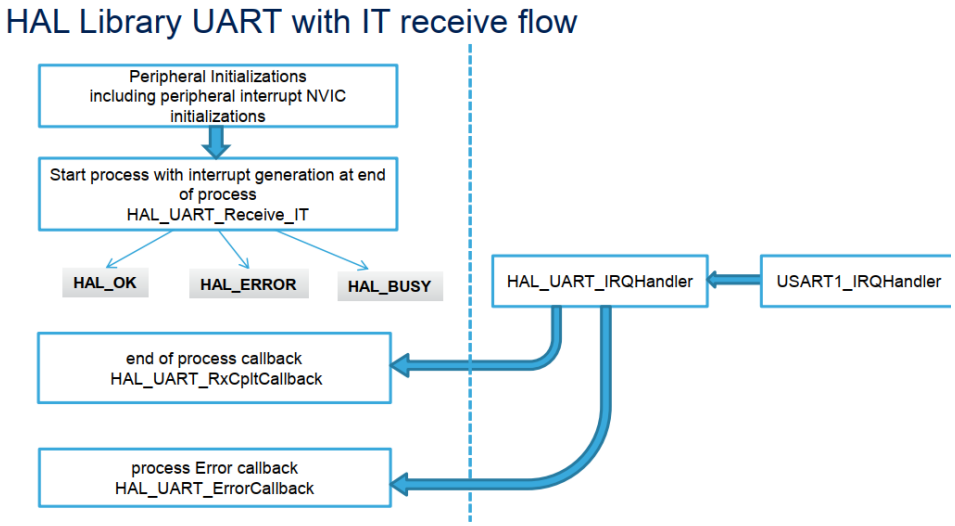


Fig 6.2 shows interrupt handling process of `USART1` module.

Normally, STM32CubeMX implements a callback function as a weak-type function by using `__weak` directive in front of function name. For example, `HAL_GPIO_EXTI_Callback()` callback function for EXTI module is implement as a weak-type function in `stm32f7xx_hal_gpio.c` file as shown in Fig 6.3. The weak-type function should not be modified but it can be reimplemented in user file, for example, `HAL_GPIO_EXTI_Callback()` function can be reimplemented in `main.c` file as shown in Fig 6.4.

```
__weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  /* Prevent unused argument(s) compilation warning */
  UNUSED(GPIO_Pin);

  /* NOTE: This function Should not be modified, when the callback is needed,
           the HAL_GPIO_EXTI_Callback could be implemented in the user file
   */
}
```

Fig 6.3 shows the callback function implemented as a weak-type function.

```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{

  if (GPIO_Pin == GPIO_PIN_13)
  {
    HAL_UART_Transmit(&huart3, (uint8_t *) "---", 3, 100);
    HAL_Delay(200);

    for(int i=0; i<20; i++)
    {
      HAL_UART_Transmit(&huart3, (uint8_t *) "B", 1, 100);
      HAL_Delay(200);
    }
  }
}
/* USER CODE END 4 */
```

Fig 6.4 shows reimplementation of EXTI callback function in `main.c file.`

Function **HAL_GPIO_EXTI_Callback ( )** in **main.c File**

- is the reimplement the EXTI callback function in `main.c` file. This is automatically invoked inside the EXTI ISR. This function has 1 argument that is EXTI number that causes the interrupt.
- To verify and acknowledge whether the interrupt was caused from pin 13.

```
if (GPIO_Pin == GPIO_PIN_13)
```

- If the interrupt signal was coming from pin 13 such as PC13 or any other pin 13 such as PA13 or PB13, MCU will print "**---**" string from UART3 and then print 20 '**B**' characters.

```
HAL_UART_Transmit(&huart3, (uint8_t *) "---", 3, 100);
HAL_Delay(200);

for(int i=0; i<20; i++)
{
      HAL_UART_Transmit(&huart3, (uint8_t *) "B", 1, 100);
      HAL_Delay(200);
}
```

7. Experiment

1. Create the new project and configure MCU pins as the following:

- Configure B1 button on PC13 as `GPIO_EXTI13` as shown in Fig 2.1 to Fig 2.3.
- Configure 3 LEDs on PB0, PB7, and PB14 pins as `GPIO_Output`.
- Configure UART3 on PD8 and PD9 pins.

Then implement ISR that responses to PC13 as shown in Fig 5.2 and implement EXTI callback function as shown in Fig 6.4.

Write the code in the infinite loop in `main()` function to send period **'.'** Indefinitely and use 400 ms for the delay time.

After that, push B1 button. Notice and record what happens in TeraTerm.



2. Connect two external buttons on PA0 and PA5 pins and then configure these buttons to generate the interrupt signal when detecting the falling edge. Then implement ISR that handles the interrupt signal from PA0 and PA5 by toggling LD2 LED on pin PB7 and LD3 LED on pin PB14.

Then modify the callback function from Experiment 1 to send **"---"** string and 20 **'E'** characters when the interrupt is from `GPIO_PIN_0`. In addition, if the interrupt is caused by `GPIO_PIN_5`, send **"---"** string and 20 **'5'** characters.

3. Experiment with priority interrupt by changing priority group to **`NVIC_PriorityGroup_2`** and configure `Preemption` and `SubPriority` as shown in Table 7.1. Use the configuration for Experiment 3.1 as shown in Table 7.1, push B1 button first and while 'B' characters are printing push external button PA0. Then do the opposite by pushing external button PA0 first and while 'E' characters are printing push B1 button. After that, change the configuration for Experiment 3.2 as shown in Table 7.1 and repeat the same process.

Table 7.1 shows interrupt priority configuration for Experiment 3

Experiment	Button	NVIC_IRQChannelPreemptionPriority	NVIC_IRQChannelSubPriority
3.1	B1	2	2
	External on PA0	2	0
3.2	B1	3	1
	External on PA0	2	3

Record the result obtained from TeraTerm for **Experiment 3.1** (push B1 button first). Specify the estimated time when push external button PA0 using down arrow (↓).

Record the result obtained from TeraTerm for **Experiment 3.1** (push external button PA0 first). Specify the estimated time when push B1 button using down arrow (↓).

Record the result obtained from TeraTerm for **Experiment 3.2** (push B1 button first). Specify the estimated time when push external button PA0 sing down arrow (↓).

Record the result obtained from TeraTerm for **Experiment 3.2** (push external button PA0 first). Specify the estimated time when push B1 button using down arrow (↓).

4. Experiment `SubPriority` by using **`NVIC_PriorityGroup_2`** and configuring `Preemption` และ `SubPriority according to` Table 7.2. For experimentation, push external button PA0 first and while 'E' characters are printing push B1 button and then external button PA5. Afterward, push external button PA0 first and while 'E' characters are printing push external button PA5 and then push B1 button. Observe the results and record.

Table 7.2 shows interrupt priority configuration for Experiment 4

Experiment	Button	NVIC_IRQChannelPreemptionPriority	NVIC_IRQChannelSubPriority
4	B1	3	1
	External PA0	2	3
	External PA5	3	3

Record the result obtained from TeraTerm for **Experiment 4** (push PA0 button first and then push B1 and PA5 buttons). Specify the estimated time when push B1 and PA5 button using down arrow (↓).

Record the result obtained from TeraTerm for **Experiment 4** (push PA0 button first and then push PA5 and B1 buttons). Specify the estimated time when push PA5 and B1 button using down arrow (↓).

Lab 4 Submission

Microcontroller Interfacing 2026

Date _____ **Lab Group No.** _____

1. Student ID _____ Name _____

2. Student ID _____ Name _____

3. Student ID _____ Name _____

Checkpoints

Experiment 2 and 3.2

Experiment 4

Question

1. From Experiment 3, if interrupt priority configuration is changed to **`NVIC_PriorityGroup_1`**, can B1 button and external button interrupt each other. If the answer is ***yes***, specify the scenario. If the answer is ***no***, specify the reason. (However, `Preemption` value 0 is reserved for pre-configured interrupts such as timer module.)